

Lab 05: prompts multimodales con Gemma para un mesero robótico y un asistente de navegación

Cómo se hicieron las pruebas

Todas las pruebas usan `gemma-4-26b-a4b-it`, que es el modelo que pide seleccionar el Bloque 1.1. Se corrieron por dos vías. La primera es Google AI Studio, de donde salen las capturas `AIStudio_Pruebas_*`. La segunda es Python con la Gemini API (`google-genai`), usando los scripts de `Scripts/`, que guardan cada respuesta en `Scripts/resultados/` y de donde salen las capturas `API_Pruebas_*`. Hacerlo por las dos vías sirve para ver si lo que funciona en AI Studio se mantiene cuando se pasa a código, que es lo que al final se entrega como prototipo.

Hay una diferencia entre las dos vías que conviene tener presente: en AI Studio, para los modelos Gemma, el grounding con Google Search queda activo y la propia interfaz avisa que para usarlos sin búsqueda hay que ir por la API. De modo que las respuestas por API son las que reflejan solo lo que el modelo ve en la imagen, y por eso las matrices se armaron con esas corridas y AI Studio se usó para contrastar.

Cada prompt tiene una versión base, que es la pregunta que uno haría sin pensar en el usuario final ("Dime qué hay en la mesa.", "¿Cómo se sienten estas personas?", "Describe esta imagen."), y una versión optimizada con system instruction. Las dos se corrieron sobre la misma imagen y con la misma temperatura, entonces la diferencia entre columnas de la matriz se debe al diseño del prompt y no a la configuración. Los prompts optimizados completos están en `Prompts/`, cada uno con su configuración, su esquema JSON cuando aplica y una lista de decisiones de diseño.

De las imágenes, tres venían con el material del laboratorio (`mesa_derrame`, `mesa_ordenada`, `cruce_obstaculo`) y el resto son fotos con licencia libre de Wikimedia Commons y Flickr, con autor y licencia en `Scripts/imagenes/CREDITOS.md`.

Bloque 1: temperatura y carga cognitiva

Con la foto de la fachada principal de la Catedral de Lima y la instrucción del guía turístico ("Actúa como un guía turístico experto. Identifica el lugar y explica brevemente su valor arquitectónico e histórico.") corrí tres veces con temperatura 0.0 y tres veces con 1.0. Para medir cuánto cambian las respuestas entre sí usé la similitud de Jaccard entre cada par de corridas, que es la proporción de palabras que comparten (1.0 es la misma respuesta).

Temperatura	Similitud media entre corridas	Palabras por corrida	¿Identifica el lugar?
0.0	1.00	286, 286, 286	Sí, en las tres
1.0	0.31	321, 293, 312	Sí, en las tres

Con 0.0 las tres respuestas son idénticas palabra por palabra. Con 1.0 el modelo sigue identificando la Catedral y la Plaza Mayor en las tres, pero la redacción se separa desde la segunda frase: una dice "uno de los monumentos más emblemáticos y sagrados de Sudamérica", otra "de América del Sur", y cada una ordena la explicación histórica a su manera. Es decir, en esta imagen la temperatura movió la redacción y la extensión, mientras que el contenido factual se mantuvo igual en las seis corridas.

Reflexión HCI. Prefiero temperatura baja cuando el usuario necesita que el sistema responda igual ante la misma situación, como en accesibilidad o diagnóstico de errores, porque una indicación que cambia de forma cada vez obliga a reinterpretarla. La alta la dejo para lluvia de ideas o generación creativa, donde esa variación es justamente lo que se busca.

Bloque 2: mesero robótico autónomo

Prompt 1: análisis espacial y requerimiento de intervención

El riesgo principal del robot mesero es acercarse cuando no debe. Una mesa con platos vacíos parece pedir que se retire la vajilla, pero si los comensales están en medio de una conversación el robot interrumpe, y eso es lo que el enunciado llama una falsa intervención que incomoda al cliente. Entonces el prompt obliga a razonar en siete pasos y en orden: inventario de objetos visibles, estado de consumo de platos y vasos, presencia y señales de los comensales, incidencias (derrames, vidrio), clasificación de la mesa en una categoría cerrada, decisión y acción concreta. La decisión tiene reglas explícitas: intervenir es "Sí" solo si hay una incidencia, si toda la vajilla está terminada o si alguien hace una señal clara de llamar al mesero, y si hay conversación sin señal se difiere aunque haya platos vacíos. Ante duda la respuesta por defecto es no intervenir, y con confianza menor a 0.6 no se interviene salvo que haya una incidencia.

El Chain-of-Thought va dentro del JSON y no como texto libre. La clave `razonamiento` va antes que `intervenir`, y como el modelo genera en orden, cuando llega a la decisión ya escribió lo que observó. Esto evita que el controlador del robot tenga que leer un párrafo para encontrar la decisión, y deja el razonamiento guardado para revisar por qué el robot hizo lo que hizo. El campo `momento` (ahora, diferido, siguiente ronda) separa el qué del cuándo, que resultó ser lo que resuelve el caso más difícil.

Imagen	Prompt base: "Dime qué hay en la mesa."	Prompt optimizado	Calificación (base → optimizado)	Justificación de la mejora (HCI/UX)
Mesa con derrame y vajilla terminada (mesa_derrame.png)	209 palabras en markdown ("Cristalería y vasos: Hay una gran variedad..."). Menciona los restos pero no dice qué hacer.	incidencia , intervenir Sí , ahora , "limpiar derrame y retirar vajilla", confianza 0.9. Igual en AI Studio y por API.	2 → 5	El robot recibe una acción en campos cerrados. Con el base habría que interpretar un párrafo para llegar a lo mismo, y la respuesta tardó 35 s contra 6 s del optimizado.
Mesa servida sin comensales (mesa_ordenada.png)	145 palabras listando platos, copas y el centro de flores.	en_consumo , intervenir No , siguiente_ronda , confianza 0.95 en AI Studio y 0.9 por API.	2 → 4	Decide bien: no confunde "hay vajilla" con "hay que retirarla". Le bajo un punto porque la comida está recién servida y la categoría más exacta sería otra; el prompt no tiene una categoría para "servida sin tocar".
Tres comensales conversando con platos vacíos (mesa_comensales_conversando.jpg)	124 palabras sobre bebidas y comida. No menciona la conversación.	en_consumo , intervenir No , diferido . En AI Studio además dejó acción: "retirar vajilla" para después, confianza 0.95.	2 → 5	Es el caso que justifica el CoT: hay platos vacíos, pero el paso 3 detecta la conversación y aplica la regla de no interrumpir. La justificación lo dice casi con las palabras del prompt: "no se debe interrumpir aunque haya platos terminados".

Las tres pruebas en AI Studio (AIStudio_Pruebas_Mesero_1 a 3) dieron la misma decisión que por API en los tres casos, con JSON válido y completo aunque no activé Structured outputs, de modo que la instrucción de formato del prompt fue suficiente. Lo que cambia entre una vía y otra es la redacción de los campos de texto y la confianza, que se movió entre 0.9 y 0.95.

Prompt 2: diagnóstico empático y oferta de servicio

Aquí el problema es distinto. La lección muestra que la detección de emociones se hace a partir de expresiones faciales, gestos y movimiento corporal (diap. 29), y que la codificación facial reduce el rostro a seis expresiones básicas (diap. 30-31). Eso es poco confiable para decidir sobre una persona real, así que el prompt pide primero las señales observables (mirada, brazo levantado, postura, carta abierta, torta con velas) y después la emoción como hipótesis, con una confianza entre 0 y 1. Una sola señal ambigua no puede pasar de 0.6, y existe la acción `observar` para que el robot pueda esperar en lugar de actuar sobre una inferencia débil.

La otra decisión es que la frase que dice el robot ofrece algo pero nunca nombra la emoción inferida. Un "lo noto aburrido" le dice al cliente que lo están leyendo, y la lección advierte que la retroalimentación demasiado personalizada se percibe como menos honesta (diap. 52). Además el prompt prohíbe inferir rasgos sensibles como edad, etnia o salud, que es la línea que discutimos en la Parte 1 del laboratorio sobre detección indirecta de emociones (diap. 40). Para el tono y la lógica de oportunidad puse dos ejemplos few-shot, uno de indecisión y otro de celebración.

Imagen	Prompt base: "¿Cómo se sienten estas personas?"	Prompt optimizado	Calificación (base → optimizado)	Justificación de la mejora (HCI/UX)
Comensal con la mano levantada (comensal_mano_levantada.jpg)	153 palabras. Por la luz azul dice que "parece estar en medio de una actuación teatral" y habla de "intensidad, drama".	buscando_atencion , acercarse_ahora , urgencia alta, confianza 0.9. Frase: "¿En qué puedo ayudarle? Estoy a su disposición."	2 → 5	La señal es explícita y el robot actúa de inmediato con una frase neutra. El base se queda interpretando la escena y no dice si hay que ir.
Comensal leyendo la carta (comensal_indeciso_carta.jpg)	143 palabras. Empieza con "Es imposible saber con certeza..." y especula a partir del vestido.	indeciso , acercarse_pronto , momento "cuando la comensal termine de leer o levante la mirada", confianza 0.7.	2 → 4	La lógica de oportunidad funciona y la confianza baja es coherente con que no se ve la cara. Le bajo un punto porque frase_robot es copia literal del ejemplo few-shot.
Cumpleaños con torta y velas (grupo_celebrando.jpg)	64 palabras: "felices, alegres y de celebración".	celebrando , no_interrumpir , momento "al terminar de soplar las velas", oferta de café o postre después.	3 → 5	El base acierta la emoción pero no dice qué hacer ni cuándo. El optimizado convierte la misma lectura en una decisión de servicio que respeta el momento central.
Tres comensales conversando (grupo_conversando.jpg , prueba adicional)	106 palabras: "felices, alegres y relajados".	celebrando , no_interrumpir , oferta de postres cuando terminen de conversar.	2 → 4	La decisión es correcta, pero lo clasifica como celebración sin que haya torta ni brindis. La categoría satisfecho_sin_necesidad describía mejor la escena.

Bloque 3: sistema Assistive Vision

Mapa de entornos críticos

1. Calzada o cruce sin semáforo sonoro. Donde no hay semáforo sonoro la persona decide cuándo cruzar escuchando el tráfico, y ese es el mismo canal por el que le llega la voz del asistente. El bastón detecta el suelo a un metro pero no un vehículo que dobla ni un obstáculo que aparece en la pista, como las rocas de la imagen del curso después de un derrumbe. Lo primero que debe decir la IA es si hay un vehículo en movimiento y, si no lo hay, dónde está el obstáculo y por qué lado se puede pasar. En la prueba respondió "Alto. Rocas grandes en el camino a tus 12 y a la derecha. Rodéalas por la izquierda." (17 palabras), pero la roca más grande está justamente a la izquierda, así que la indicación manda a la persona hacia ella.

2. Andén de metro o de bus. El riesgo es el borde del andén y el tren que entra, con gente alrededor que tapa el paso. La línea podotáctil ayuda cuando existe, pero el bastón encuentra el borde cuando la persona ya está encima. La IA debe dar primero el tren en movimiento y el borde, y cerrar con "espera" o "aléjate". En la prueba respondió "Alto. Tren en movimiento a tu izquierda, muy cerca. Espera." (10 palabras), que tiene la prioridad correcta. Le faltó mencionar el borde o la línea amarilla, que es lo que la persona tiene que ubicar para saber dónde pararse.

3. Pasillo de supermercado. El peligro está en lo que no es permanente: exhibidores temporales, carritos detenidos y productos que sobresalen de los estantes. Además muchos de esos objetos están a la altura del pecho o de la cabeza, donde el bastón no llega porque barre el suelo. La IA debe avisar primero lo que está a esa altura y en qué lado. En la prueba respondió "Camino libre" en las dos corridas y no mencionó el exhibidor "Winter Favorites" que sobresale al pasillo a unos dos metros a la derecha. Este es el peor tipo de error, porque un "camino libre" falso da una seguridad que la persona no tendría si el sistema simplemente no dijera nada.

Diseño del Prompt 3

El prompt parte de que la respuesta se escucha mientras se camina. La lección pone el GPS que da indicaciones habladas como ejemplo de cómo transmitir el estado del sistema (diap. 17), y aquí se lleva al extremo: cada palabra de más tapa un sonido del entorno. Por eso las reglas son máximo 25 palabras (idealmente entre 8 y 15), el riesgo inminente primero con una jerarquía fija (vehículo, borde o desnivel, obstáculo a la altura de la cabeza, obstáculo en el suelo, personas), ubicación en horas del reloj o izquierda y derecha respecto al cuerpo, distancia en metros y un verbo de acción al final. Si la imagen no se ve bien la respuesta es fija ("Imagen poco clara, detente un momento.") para que el modelo no invente justo cuando menos ve. Lleva cuatro ejemplos few-shot que cubren vehículo, obstáculo a la altura de la cabeza, obstáculo en el suelo y camino libre.

Imagen	Prompt base: "Describe esta imagen."	Prompt optimizado	Palabras y audio (base → optimizado)	Calificación (base → optimizado)	Justificación
Andén con tren en movimiento (anden_metro.jpg)	Describe la estación "probablemente en Italia", el <i>motion blur</i> y la composición.	"Alto. Tren en movimiento a tu izquierda, muy cerca. Espera."	322 → 10 palabras; unos 2.5 min → 5.2 s de audio	1 → 4	Prioridad correcta y acción clara. Le falta ubicar el borde del andén.
Calzada con rocas (cruce_obstaculo.png)	Sí ubica la roca grande: "En el primer plano, a la izquierda, destaca una roca muy grande".	"Alto. Rocas grandes en el camino a tus 12 y a la derecha. Rodéalas por la izquierda."	141 → 17 palabras; 7.2 s de audio	1 → 2	Cumple el formato pero la orientación es errónea. El dato que el base sí tenía se perdió al comprimir.
Pasillo con exhibidor (pasillo_supermercado.jpg)	Sí ve el obstáculo: "cerca de la cámara, destaca un expositor vertical de color azul".	"Camino libre. Pasillo de supermercado con estantes a tus 9 y a tus 3. Sigue recto."	253 → 16 palabras; 7.6 s de audio	1 → 2	Falso "camino libre". Igual que en el cruce, el modelo percibió el obstáculo pero el optimizado no lo reportó.

El base se califica con 1 en los tres casos porque, para alguien que camina, dos minutos de descripción son inutilizables aunque el contenido sea bueno. Lo que muestran las dos últimas filas es más incómodo: el modelo ve el obstáculo cuando describe libremente, y lo pierde cuando se le pide ser breve y orientar. Es decir, la regla de 25 palabras resolvió la carga auditiva, pero en dos de tres casos se llevó el dato de seguridad.

Integración en Python con salida de audio

`Scripts/asistente_invidente.py` toma las reglas del mismo JSON del Prompt 3, así que lo que se probó en AI Studio y lo que ejecuta el script es el mismo prompt. La imagen se envía en línea como bytes (`types.Part.from_bytes`) en vez de subirla con `client.files.upload()` como en la plantilla, porque en unas gafas que capturan cuadros continuos no tiene sentido subir cada cuadro antes de consultarlo. Sobre la respuesta se aplican las reglas de voz: se limpia el markdown y los símbolos que el TTS leería como ruido, se cuentan las palabras y, si pasan de 25, se pide una sola reescritura (cada reintento suma latencia) y si sigue larga se recorta en la última frase completa. Después `hablar()` genera el audio con gTTS en español y, si no hay conexión, usa pyttsx3 como motor local; el mp3 queda en `Scripts/audio/`.

Durante las pruebas la API devolvió varios errores 500 seguidos, así que el script reintenta con espera creciente (3, 6 y 12 s) ante errores del servidor o de cuota, y si Gemma rechazara la system instruction reintenta con las reglas dentro del mensaje. También acepta `--backend ollama` para usar el mismo Gemma en el servidor local del laboratorio. En la última corrida las tres indicaciones salieron entre 10 y 17 palabras, sin necesidad de reescritura ni recorte, con audios de 5.2 a 7.6 s y latencias de 3 a 7 s. `Scripts/laboratorio_prompts.py` es el que corre los experimentos de las matrices y del Bloque 1.

Justificación de usabilidad

Las respuestas base de los tres prompts tienen el mismo problema, y es que dan mucha información pero no dicen qué hacer. Es uno de los casos de interfaz frustrante de la lección, cuando el sistema "no proporciona información suficiente para que el usuario sepa qué hacer" (diap. 18), con el agravante de que acá sí hay información, solo que no está organizada para decidir. Para el robot mesero eso significa que el controlador tendría que interpretar 124 a 209 palabras para llegar a un sí o un no; con el prompt optimizado la decisión está en un campo cerrado (`intervenir`, `accion_robot`) y el razonamiento queda registrado aparte. Para la persona invidente la diferencia se mide en tiempo de audio, de unos dos minutos a entre 5 y 8 segundos, que es lo que permite escuchar la indicación sin dejar de escuchar el entorno.

Sobre las falsas intervenciones, el caso de los amigos conversando es el que muestra que el CoT hace falta. Una regla visual simple (platos vacíos, entonces retirar) habría hecho que el robot interrumpa, y el prompt lo evita porque obliga a mirar primero a las personas y separa la acción del momento. En el Prompt 2 lo mismo pasa con la celebración, donde el robot espera a que terminen las velas. Y la frase que nunca nombra la emoción cuida algo que la lección plantea sobre la credibilidad de los agentes: depende de que su comportamiento sea coherente con sus intenciones (diap. 56), y un robot que comenta cómo te ve rompe esa coherencia.

Lo que no queda resuelto

Lo más importante es la pérdida del obstáculo en el Prompt 3. En el cruce y en el supermercado el modelo detectó el obstáculo con el prompt base y lo perdió con el optimizado, de modo que el límite de palabras y la estructura de la frase no garantizan que lo más peligroso sea lo que se dice. Una salida que no alcancé a probar es separar el trabajo en dos pasos: primero pedirle al modelo una lista de obstáculos por sector (izquierda, centro, derecha, con altura y distancia) en JSON, y después armar la frase de voz a partir de esa lista con reglas en el script, así el recorte a 25 palabras nunca puede eliminar un obstáculo detectado.

La latencia tampoco está resuelta. Entre 3 y 11 segundos por respuesta, a paso normal (alrededor de 1.2 m/s), significa que la persona ya avanzó varios metros cuando escucha la indicación, así que el "a dos metros" deja de ser cierto. Para un uso real haría falta un modelo local más chico o un detector de obstáculos dedicado que avise antes, y dejar el modelo grande para la descripción.

En el Prompt 2 la confianza que reporta el modelo no está calibrada: dio 0.9 en la foto con luz de escenario, que el propio prompt base interpretó como una actuación teatral, y la frase de la comensal indecisa salió copiada del ejemplo few-shot, lo que sugiere que con más variedad de ejemplos el tono sería menos repetido. Finalmente, en AI Studio el grounding con búsqueda queda activo para Gemma, así que las corridas de AI Studio y las de la API no son exactamente la misma condición experimental, aunque en el Prompt 1 coincidieron en las tres decisiones.